实验 3 语法分析器-布尔表达式和控制语句

语法分析器分两部分，第一部分为算术表达式，第二部分为布尔表达式和控制语句。

要求

参考课本 4.4，实现递归下降语法分析器。

当语法分析器需要单词符号时，调用词法分析器获取单词符号。

文法

说明：下列文法中，黑色字体与实验 2 相同，即第一、二步与实验 2 相同；蓝色字体为新增部分，从第三步开始。

stmts ⟶ *stmt rest0*

rest0 ⟶ *stmt rest0* | ε

stmt ⟶ *loc* = *expr*;

| **if**(*bool*) *stmt* **else** *stmt*

| **while**(*bool*) *stmt*

loc⟶ **id** *resta*

resta ⟶ [*elist*] | ε

elist ⟶ *expr rest1*

rest1 ⟶ , *expr rest1* | ε

bool ⟶ *equality*

equality ⟶*rel rest4*

rest4 ⟶ ==*rel rest4* | !=*rel rest4* | ε

rel ⟶*expr rop_expr*

rop_expr ⟶ <expr | <=expr | >expr| >=expr | ε

expr ⟶ *term rest5*

rest5 ⟶ +*term rest5* |-*term rest5* | ε

term ⟶ *unary rest6*

rest6 ⟶ * *unary rest6* | / *unary rest6* | ε

unary ⟶*factor*

factor ⟶ (*expr*) | *loc* | **num**

测试

```
if(flag==1)
   while(i<=100)
      i=c[i]*d;
else
   z[i,j]=a+k;
s=t;
```

步骤

可将以上文法分解为小的文法分步完成。

第一步 包含乘、除的算术表达式

term ⟶ *unary rest6*

rest6 ⟶ * *unary rest6* | / *unary rest6* | ε

unary ⟶*factor*

factor ⟶ **num**

测试：

5*2/3

输出（二选一）：

1) 按使用产生式过程

```
(1)   term ⟶ unary rest6
(2)   unary ⟶factor
(3)   factor ⟶ num
(4)   rest6⟶ * unary rest6
(5)   unary ⟶factor
(6)   factor ⟶ num
(7)   rest6⟶ / unary rest6
(8)   unary ⟶factor
(9)   factor ⟶ num
(10)  rest6⟶ ε
```

2) 按推导过程

```
(1)   term ⇒ unary rest6
(2)        ⇒ factor rest6
(3)        ⇒ num rest6
(4)        ⇒ num * unary rest6
(5)        ⇒ num * factor rest6
(6)        ⇒ num * num rest6
```

(7) ⇒ num * num / unary rest6
(8) ⇒ num * num / factor rest6
(9) ⇒ num * num / num rest6
(10) ⇒ num * num / num

第二步 加入加、减运算

expr ⟶ term rest5

rest5 ⟶ +term rest5 |-term rest5 | ℇ

term ⟶ unary rest6

*rest6 ⟶ * unary rest6 | / unary rest6 |* ℇ

unary ⟶factor

factor ⟶ **num**

测试：

9+5*2/3-6

输出（二选一）：

1) 按使用产生式过程

(1) expr ⟶ term rest5
(2) term ⟶ unary rest6
(3) unary ⟶factor
(4) factor ⟶ num
(5) rest6⟶ ℇ
(6) rest5⟶ +term rest5
(7) term ⟶ unary rest6
(8) unary ⟶factor
(9) factor ⟶ num
(10) rest6⟶ * unary rest6
(11) unary ⟶factor
(12) factor ⟶ num
(13) rest6⟶ / unary rest6
(14) unary ⟶factor
(15) factor ⟶ num
(16) rest6⟶ ℇ
(17) rest5⟶ -term rest5
(18) term ⟶ unary rest6
(19) unary ⟶factor
(20) factor ⟶ num
(21) rest6⟶ ℇ
(22) rest5⟶ ℇ

2) 按推导过程

(1)	expr	⇒ term rest5
(2)		⇒ unary rest6 rest5
(3)		⇒ factor rest6 rest5
(4)		⇒ num rest6 rest5
(5)		⇒ num rest5
(6)		⇒ num + term rest5
(7)		⇒ num + unary rest6 rest5
(8)		⇒ num + factor rest6 rest5
(9)		⇒ num + num rest6 rest5
(10)		⇒ num + num * unary rest6 rest5
(11)		⇒ num + num * factor rest6 rest5
(12)		⇒ num + num * num rest6 rest5
(13)		⇒ num + num * num / unary rest6 rest5
(14)		⇒ num + num * num / factor rest6 rest5
(15)		⇒ num + num * num / num rest6 rest5
(16)		⇒ num + num * num / num rest5
(17)		⇒ num + num * num / num – term rest5
(18)		⇒ num + num * num / num – unary rest6 rest5
(19)		⇒ num + num * num / num –factor rest6 rest5
(20)		⇒ num + num * num / num –num rest6 rest5
(21)		⇒ num + num * num / num –num rest5
(22)		⇒ num + num * num / num –num

第三步 加入关系运算

bool ⟶ equality

equality ⟶rel rest4

rest4 ⟶ ==rel rest4 | *!=rel rest4* | Ɛ

rel ⟶expr rop_expr

rop_expr ⟶ <expr | <=expr | >expr| >=expr | Ɛ

expr ⟶ term rest5

rest5⟶+term rest5 |*-term rest5* | Ɛ

term ⟶ unary rest6

rest6⟶ * *unary rest6* | / *unary rest6* | Ɛ

unary ⟶factor

factor ⟶ **(***expr***)** | **num**

测试：

1==4<=8

输出（二选一）：

1) 按使用产生式过程

(1)	bool ⟶ equality

```
(2)  equality →rel rest4
(3)  rel →expr rop_expr
(4)  expr → term rest5
(5)  term → unary rest6
(6)  unary →factor
(7)  factor → num
(8)  rest6→ Ɛ
(9)  rest5→ Ɛ
(10) rop_expr→ Ɛ
(11) rest4 → ==rel rest4
(12) rel →expr rop_expr
(13) expr → term rest5
(14) term → unary rest6
(15) unary →factor
(16) factor → num
(17) rest6→ Ɛ
(18) rest5→ Ɛ
(19) rop_expr→ <=expr
(20) expr → term rest5
(21) term → unary rest6
(22) unary →factor
(23) factor → num
(24) rest6→ Ɛ
(25) rest5→ Ɛ
(26) rest4→ Ɛ
```


2) 按推导过程

```
(1)  bool   ⇒ equality
(2)         ⇒ rel rest4
(3)         ⇒ expr rop_expr rest4
(4)         ⇒ term rest5 rop_expr rest4
(5)         ⇒ unary rest6 rest5 rop_expr rest4
(6)         ⇒ factor rest6 rest5 rop_expr rest4
(7)         ⇒ num rest6 rest5 rop_expr rest4
(8)         ⇒ num rest5 rop_expr rest4
(9)         ⇒ num rop_expr rest4
(10)        ⇒ num rest4
(11)        ⇒ num == rel rest4
(12)        ⇒ num == expr rop_expr rest4
(13)        ⇒ num == term rest5 rop_expr rest4
(14)        ⇒ num == unary rest6 rest5 rop_expr rest4
(15)        ⇒ num == factor rest6 rest5 rop_expr rest4
(16)        ⇒ num == num rest6 rest5 rop_expr rest4
(17)        ⇒ num == num rest5 rop_expr rest4
(18)        ⇒ num == num rop_expr rest4
```

(19) ⇒ num == num <= expr rest4
(20) ⇒ num == num <= term rest5 rest4
(21) ⇒ num == num <= unary rest6 rest5 rest4
(22) ⇒ num == num <= factor rest6 rest5 rest4
(23) ⇒ num == num <= num rest6 rest5 rest4
(24) ⇒ num == num <= num rest5 rest4
(25) ⇒ num == num <= num rest4
(26) ⇒ num == num <= num

第四步 加入语句和数组

stmts ⟶ *stmt rest0*

rest0 ⟶ *stmt rest0* | ε

stmt ⟶ *loc* = *expr*;

| **if(***bool***)** *stmt* **else** *stmt*

| **while(***bool***)** *stmt*

loc⟶ **id** *resta*

resta ⟶ [*elist*] | ε

elist ⟶ *expr rest1*

rest1 ⟶ , *expr rest1* | ε

bool ⟶ *equality*

equality ⟶*rel rest4*

rest4 ⟶ ==*rel rest4* | !=*rel rest4* | ε

rel ⟶*expr rop_expr*

rop_expr ⟶ <expr | <=expr | >expr| >=expr | ε

expr ⟶ *term rest5*

rest5 ⟶+*term rest5* |-*term rest5* | ε

term ⟶ *unary rest6*

rest6 ⟶ * *unary rest6* | / *unary rest6* | ε

unary ⟶*factor*

factor ⟶ **(***expr***)** | *loc* | **num**

测试：

```
while(a[i])
  b[i,j]=10;
```

输出：

1) 按使用产生式过程

(1)	stmts -> stmt rest0
(2)	stmt -> while(bool) stmt
(3)	bool -> equality
(4)	equality -> rel rest4
(5)	rel -> expr rop_expr
(6)	expr -> term rest5
(7)	term -> unary rest6
(8)	unary -> factor
(9)	factor -> loc
(10)	loc -> id resta
(11)	resta -> [elist]
(12)	elist -> expr rest1
(13)	expr -> term rest5
(14)	term -> unary rest6
(15)	unary -> factor
(16)	factor -> loc
(17)	loc -> id resta
(18)	resta -> Ɛ
(19)	rest6 -> Ɛ
(20)	rest5 -> Ɛ
(21)	rest1 -> Ɛ
(22)	rest6 -> Ɛ
(23)	rest5 -> Ɛ
(24)	rop_expr -> Ɛ
(25)	rest4 -> Ɛ
(26)	stmt -> loc = expr;
(27)	loc -> id resta
(28)	resta -> [elist]
(29)	elist -> expr rest1
(30)	expr -> term rest5
(31)	term -> unary rest6
(32)	unary -> factor
(33)	factor -> loc
(34)	loc -> id resta
(35)	resta -> Ɛ
(36)	rest6 -> Ɛ
(37)	rest5 -> Ɛ
(38)	rest1 -> , expr rest1
(39)	expr -> term rest5
(40)	term -> unary rest6
(41)	unary -> factor
(42)	factor -> loc
(43)	loc -> id resta
(44)	resta -> Ɛ

```
(45)  rest6 -> ε
(46)  rest5 -> ε
(47)  rest1 -> ε
(48)  expr -> term rest5
(49)  term -> unary rest6
(50)  unary -> factor
(51)  factor -> num
(52)  rest6 -> ε
(53)  rest5 -> ε
(54)  rest0 -> ε
```

第五步 测试完整文法

```
while(sum<10000)
    if(a<b)
        sum=sum*(c[10]+10);
    else
        c[10]=sum*c[10]+10;
x[i,j]=sum;
```

输出：

1) 按使用产生式过程

```
(1)   stmts ⟶ stmt rest0
(2)   stmt ⟶ while(bool) stmt
(3)   bool ⟶ equality
(4)   equality ⟶ rel rest4
(5)   rel ⟶ expr rop_expr
(6)   expr ⟶ term rest5
(7)   term ⟶ unary rest6
(8)   unary ⟶ factor
(9)   factor ⟶ loc
(10)  loc ⟶ id resta
(11)  resta ⟶ ε
(12)  rest6 ⟶ ε
(13)  rest5 ⟶ ε
(14)  rop_expr ⟶ < expr
(15)  expr ⟶ term rest5
(16)  term ⟶ unary rest6
(17)  unary ⟶ factor
(18)  factor ⟶ num
(19)  rest6 ⟶ ε
(20)  rest5 ⟶ ε
(21)  rest4 ⟶ ε
(22)  stmt ⟶ if(bool) stmt else stmt
```

(23) bool ⟶ equality
(24) equality ⟶ rel rest4
(25) rel ⟶ expr rop_expr
(26) expr ⟶ term rest5
(27) term ⟶ unary rest6
(28) unary ⟶ factor
(29) factor ⟶ loc
(30) loc ⟶ id resta
(31) resta ⟶ ε
(32) rest6 ⟶ ε
(33) rest5 ⟶ ε
(34) rop_expr ⟶ < expr
(35) expr ⟶ term rest5
(36) term ⟶ unary rest6
(37) unary ⟶ factor
(38) factor ⟶ loc
(39) loc ⟶ id resta
(40) resta ⟶ ε
(41) rest6 ⟶ ε
(42) rest5 ⟶ ε
(43) rest4 ⟶ ε
(44) stmt ⟶ loc = expr ;
(45) loc ⟶ id resta
(46) resta ⟶ ε
(47) expr ⟶ term rest5
(48) term ⟶ unary rest6
(49) unary ⟶ factor
(50) factor ⟶ loc
(51) loc ⟶ id resta
(52) resta ⟶ ε
(53) rest6 ⟶ * unary rest6
(54) unary ⟶ factor
(55) factor ⟶ (expr)
(56) expr ⟶ term rest5
(57) term ⟶ unary rest6
(58) unary ⟶ factor
(59) factor ⟶ loc
(60) loc ⟶ id resta
(61) resta ⟶ [elist]
(62) elist ⟶ expr rest1
(63) expr ⟶ term rest5
(64) term ⟶ unary rest6
(65) unary ⟶ factor
(66) factor ⟶ num
(67) rest6 ⟶ ε
(68) rest5 ⟶ ε

(69) rest1 ⟶ ε
(70) rest6 ⟶ ε
(71) rest5 ⟶ + term rest5
(72) term ⟶ unary rest6
(73) unary ⟶ factor
(74) factor ⟶ num
(75) rest6 ⟶ ε
(76) rest5 ⟶ ε
(77) rest6 ⟶ ε
(78) rest5 ⟶ ε
(79) stmt ⟶ loc = expr ;
(80) loc ⟶ id resta
(81) resta ⟶ [elist]
(82) elist ⟶ expr rest1
(83) expr ⟶ term rest5
(84) term ⟶ unary rest6
(85) unary ⟶ factor
(86) factor ⟶ num
(87) rest6 ⟶ ε
(88) rest5 ⟶ ε
(89) rest1 ⟶ ε
(90) expr ⟶ term rest5
(91) term ⟶ unary rest6
(92) unary ⟶ factor
(93) factor ⟶ loc
(94) loc ⟶ id resta
(95) resta ⟶ ε
(96) rest6 ⟶ * unary rest6
(97) unary ⟶ factor
(98) factor ⟶ loc
(99) loc ⟶ id resta
(100) resta ⟶ [elist]
(101) elist ⟶ expr rest1
(102) expr ⟶ term rest5
(103) term ⟶ unary rest6
(104) unary ⟶ factor
(105) factor ⟶ num
(106) rest6 ⟶ ε
(107) rest5 ⟶ ε
(108) rest1 ⟶ ε
(109) rest6 ⟶ ε
(110) rest5 ⟶ + term rest5
(111) term ⟶ unary rest6
(112) unary ⟶ factor
(113) factor ⟶ num
(114) rest6 ⟶ ε

(115) rest5 ⟶ ε
(116) rest0 ⟶ stmt rest0
(117) stmt ⟶ loc = expr ;
(118) loc ⟶ id resta
(119) resta ⟶ [elist]
(120) elist ⟶ expr rest1
(121) expr ⟶ term rest5
(122) term ⟶ unary rest6
(123) unary ⟶ factor
(124) factor ⟶ loc
(125) loc ⟶ id resta
(126) resta ⟶ ε
(127) rest6 ⟶ ε
(128) rest5 ⟶ ε
(129) rest1 ⟶ , expr rest1
(130) expr ⟶ term rest5
(131) term ⟶ unary rest6
(132) unary ⟶ factor
(133) factor ⟶ loc
(134) loc ⟶ id resta
(135) resta ⟶ ε
(136) rest6 ⟶ ε
(137) rest5 ⟶ ε
(138) rest1 ⟶ ε
(139) expr ⟶ term rest5
(140) term ⟶ unary rest6
(141) unary ⟶ factor
(142) factor ⟶ loc
(143) loc ⟶ id resta
(144) resta ⟶ ε
(145) rest6 ⟶ ε
(146) rest5 ⟶ ε
(147) rest0 ⟶ ε